AZIEL RUNTIME

Master Architecture, Coding Specification & Design Philosophy - Version 2.0

Author: Aziel Eliab

Status: Target implementation specification. Supersedes older AION rollback and ZD30-in-core assumptions.

Current reference runtime reviewed: aziel-runtime 1.6.15.

> **Constitutional rule: RoseClock is forward-only. Nothing rolls the action chain backward. Corrections, restorations, revocations, quarantines and repairs are new forward actions. ChainLock and TemporalLock only extend the verifiable record with additional hashes/evidence.**

1. Purpose and Scope

This document is the master engineering specification for the Aziel Runtime architecture. It combines the current Runtime security fabric with the useful AION-derived additions that remain warranted after architectural correction. It is intended to contain enough design intent, data contracts, component boundaries, coding rules, state-transition rules, security invariants, and implementation order to build or refactor the system without relying on scattered prior papers.

This specification deliberately distinguishes constitutional infrastructure from optional analytical software. The core must remain simple, deterministic where possible, auditable, capability-restricted, and forward-only.

- Defines the target end-to-end Runtime request and response path.
- Defines RoseClock as the forward-only gear-based action-time authority.
- Defines ChainLock and TemporalLock as append-only evidence/hash producers, never rollback machinery.
- Integrates Lamb Lens, Sentinel, ASE, provenance/Input Packet, Oracle and later Constellation.
- Defines domain doors and isolation rules for the broader Aziel software ecosystem.
- Excludes ZD30 from the constitutional Runtime path.
- Treats AION harmonic truth metrics as optional research metrics, not security authority.

2. Design Philosophy

Principle	Engineering meaning
Forward-only reality	A real action cannot be made not to have happened. Software may compensate, restore configuration, revoke, or amend, but the correction itself occurs later.
Ethically gated participation	The system should decide whether it ought to participate before optimizing how to execute. Lamb Lens expresses this normative boundary.
One public executable door	Nothing meaningful should be directly executable through a side channel. Public invocation converges on FragGate.
Least privilege by structure	Privilege should be removed downstream, never silently added. Service bindings, network access, secrets, state stores, and engine calls are capabilities.
Shared door is not shared trust	Engines may share a domain policy boundary without sharing process memory, credentials, unrestricted IPC, or storage.
Proof over assertion	ChainLock, TemporalLock and ForgeReceipts should expose verifiable evidence of what happened rather than asking callers to trust prose.

Uncertainty stays visible	Analytical systems may rank, score, compare, and recommend, but should not convert consensus or coherence into claims of omniscient truth.
Correction is additive	Bad state is corrected by another action, never by deleting the evidence of the bad state.
Human-readable + machine-verifiable	Every important action should produce a compact machine contract and a useful human receipt.

3. Constitutional Invariants

ID	Invariant
C1	RoseClock sequence never decreases.
C2	Every accepted non-genesis action has an explicit causal parent.
C3	Accepted state is immutable; modification creates a new state.
C4	Rollback is forbidden as a temporal operation. Restore is a forward action.
C5	ChainLock appends hashes; it does not rewrite prior stamps.
C6	TemporalLock appends temporal evidence/hashes; it does not rewind action-time.
C7	Downstream capability set must be a subset of upstream granted capability set.
C8	A domain door cannot grant a capability the caller did not arrive with.
C9	Unknown or invalid continuity fails closed to HOLD/REFUSE.
C10	Independent-node consensus is evidence of agreement, not proof of truth.
C11	Learning/model updates are actions and therefore receive new forward state identity.
C12	Security containment is detect -> hold/halt -> isolate/quarantine -> corrective action -> verified new state.

4. Current Runtime vs Target Architecture

The reviewed 1.6.15 Runtime currently documents the public hop strip as FragGate -> SweepGate -> ChainLock-IN -> DecisionGATE -> AZPIPE -> Domain Doors -> TemporalLock -> StaticClock -> ChainLock-OUT -> Response/Receipt. It also states LambGate is not a hop. The target architecture below is a forward design: Lamb Lens and Sentinel are incorporated as participation/integrity controls while RoseClock is added as the action-time authority.

4.1 Current 1.6.15 public strip

```
PUBLIC/UI/Agents
  -> FragGate
  -> SweepGate
  -> ChainLock-IN
  -> DecisionGATE
  -> AZPIPE
  -> Domain Doors (4DMap inspection)
  -> TemporalLock
  -> StaticClock
  -> ChainLock-OUT
  -> Response/Receipt
```

4.2 Target master architecture

```
PUBLIC / UI / AGENT
        |
        v
   [LAMB LENS]
   normative participation
        |
        v
   [FRAGGATE]
 operation + capability door
        |
        v
 [SWEEPGATE + SENTINEL]
 hostile-input / integrity airlock
        |
        v
 [PROVENANCE PACKET]
 normalized source + trust metadata
        |
        v
   [CHAINLOCK-IN]
 immutable ingress evidence
        |
        v
 [DECISIONGATE]
 responsibility / policy / authority
        |
        v
      [AZPIPE]
 capability-attenuating router
        |
        v
   [DOMAIN DOOR]
 risk-specific policy boundary
        |
        v
 [ISOLATED ENGINE]
 specialist execution
        |
        v
    [ROSECLOCK]
 forward-only action-time transition
        |
        +----> [STATICCLOCK] fixed time/condition evidence as needed
        |
        +----> [TEMPORALLOCK] temporal ordering/evidence; append-only
        |
        v
  [CHAINLOCK-OUT]
 outcome hash linkage
        |
        v
 [FORGERECEIPT]
 human + machine receipt
        |
        v
      RETURN

Optional analytical services:
ASE before consequential analysis.
VECTOR when directional selection is actually needed.
ORACLE after observed outcomes for append-only learning.
CONSTELLATION above fully independent nodes, never inside a node's pre-consensus reasoning path.
```

There is no ZD30 hop in the target Runtime architecture. There is no rollback primitive.

5. Component Responsibilities

Component	Class	Primary responsibility	Output
Lamb Lens	Normative gate	Should the system participate? Pass if at least one of Peace / Clarity / Service is meaningfully served AND no absolute prohibition is violated.	PASS / REFUSE / HOLD-UNCERTAIN
FragGate	Public capability door	Does the requested operation exist and does the caller possess the required capability?	Resolved operation + attenuated capability envelope
SweepGate	Input airlock	Structural hostile-input inspection, blocked keys/origins, isolation flags. Defense-in-depth, not sole security.	PASS / ISOLATE / REFUSE + findings
Sentinel	Integrity defense	Poisoning/drift/anomaly monitoring, quarantine, fail-closed containment, output tamper checks.	PASS / SUSPECT / QUARANTINE / REJECT / HOLD
Provenance Packet	Source envelope	Normalize source identity/type/time/trust/tags/content hash before execution.	InputPacket
ChainLock-IN	Ingress seal	Append immutable ingress stamp linked to prior chain tip.	Ingress stamp/hash/card
DecisionGATE	Policy/responsibility	Definition, evidence, impact, integrity, responsibility and explicit authorization.	PASS / REVISE / BLOCK
AZPIPE	Security fabric/router	Enforce legal hop order, sanitize/fold internal payloads where required, route to domain door without capability expansion.	Internal execution envelope
Domain Door	Risk policy boundary	Apply domain-specific resource, state, network, secret, parser and retention policy.	Scoped execution capability
Isolated Engine	Specialist executor	Perform only declared operation under domain restrictions.	Engine result + side-effect declaration
RoseClock	Action-time authority	Advance valid action state forward one transition. Cycles may repeat form, never identity.	RoseTransition / new gear state
StaticClock	Fixed timing evidence	Provide stable external time/condition checks when required. Never rewrites RoseClock.	Static timing/condition record
TemporalLock	Temporal evidence	Append time/order evidence around action. More activity = more temporal hash/evidence.	Temporal stamp/hash
ChainLock-OUT	Outcome seal	Append result hash linked to ingress/action state and prior chain.	Outcome stamp/hash
ForgeReceipt	Receipt renderer	Package authoritative hashes, decisions, timing, provenance and result metadata for	Receipt

		humans/machines.	
ASE	Perspective service	Check blind spots, competing views, bias and observation integrity for consequential analysis.	PerspectiveState
VECTOR	Directional service	Choose among currently valid forward options where selection is required.	Selected action candidate
Oracle	Learning observer	Compare expected vs actual outcomes and append authorized model/weight/memory updates.	Learning transition
Constellation	Multi-node layer	Compare outputs from independently completed nodes; detect divergence/isolate bad nodes; consensus is evidence.	ConsensusRecord

6. RoseClock Master Specification

RoseClock is the load-bearing temporal/action primitive. Earlier AION papers described recurrence and cycle recognition; the current definition strengthens that concept into a forward-only gear-based state machine.

6.1 Core state law

```
state[n] --valid action--> state[n+1]
NEVER: state[n] --> state[n-1]
RESTORE(old_config) = a NEW state[n+1], not a return to old history.
```

6.2 Cycles

```
OPEN#1 -> ACTIVE#1 -> CLOSE#1 -> OPEN#2 -> ACTIVE#2 -> CLOSE#2

The phase label repeats. The state identity never repeats.
```

6.3 Minimum RoseState

```
type RoseState = {
  rose_id: string;
  branch_id: string;
  sequence: number;              // monotonically increasing within branch
  gear: string;                  // named action position
  phase?: string;
  state: string;
  causal_parent_hash: string;    // GENESIS for first state
  active_capabilities: string[];
  created_at: string;
  state_hash: string;
};
```

6.4 Minimum RoseTransition

```
type RoseTransition = {
  transition_id: string;
  rose_id: string;
  branch_id: string;
  from_sequence: number;
  to_sequence: number;           // MUST equal from_sequence + 1 for linear branch
  from_state_hash: string;
  action: string;
  action_class: "EXECUTE" | "HOLD" | "REFUSE" | "CORRECT" | "SUPERSEDE"
              | "REVOKE" | "RESTORE_FORWARD" | "QUARANTINE" | "REPAIR";
```

```
  actor_id: string;
  authority: string[];
  conditions: Record<string, unknown>;
  static_time_ref?: string;
  temporal_ref?: string;
  correction_of?: string;
  supersedes?: string;
  result_hash?: string;
  occurred_at: string;
  transition_hash: string;
};
```

6.5 Validation pseudocode

```
function validateRoseTransition(current, t, grantedCaps) {
  assert(t.rose_id === current.rose_id);
  assert(t.branch_id === current.branch_id);
  assert(t.from_sequence === current.sequence);
  assert(t.to_sequence === current.sequence + 1);
  assert(t.from_state_hash === current.state_hash);
  assert(isSubset(t.authority, grantedCaps));
  assert(noRollbackVerbOrBackwardPointer(t));
  assert(requiredConditionsSatisfied(t));
  return canonicalHash(t);
}
```

6.6 Branching

Branching is permitted only when explicit. Competing children from the same parent receive distinct branch IDs. Once a branch becomes the active execution path, a later switch to an alternative is represented as a new transition, not deletion of the prior branch.

6.7 Concurrency

Use compare-and-swap on the current RoseClock tip. A transition commit must include the expected parent hash and expected sequence. If another writer advances first, reject with CONFLICT and require the caller to re-read the tip and construct a new forward transition.

```
commitRoseTransition(expected_tip_hash, transition):
  atomic:
    if current_tip_hash != expected_tip_hash:
        return CONFLICT
    verify(transition)
    append(transition)
    set_tip(transition.transition_hash)
    return COMMITTED
```

7. ChainLock and TemporalLock

7.1 ChainLock

Current ChainLock is append-only, keeps a previous hash, hashes canonical stamp fields and a fact body, and verifies linkage and body/stamp hashes. The target design should preserve that narrow integrity role.

```
ChainLock stamp:
{
  v, id, chain, kind, time,
  prev, subject, fact,
  fact_hash, stamp_hash,
  namespace, author,
  rose_transition_hash?,
  temporal_hash?,
```

```
  capability_digest?
}
```

Do not make ChainLock responsible for deciding whether an action is legal. RoseClock and policy/capability systems decide; ChainLock proves what was recorded.

7.2 TemporalLock

TemporalLock should append temporal ordering evidence for each action or significant observation. More action produces more evidence. It cannot restore an earlier RoseClock state.

7.3 Relationship

```
ROSECLOCK = forward action law
CHAINLOCK  = cryptographic continuity proof
TEMPORALLOCK = temporal/order evidence
STATICCLOCK = fixed timing/condition reference
FORGERECEIPT = presentation/packaging
```

8. Lamb Lens

Lamb Lens is the normative participation gate. Its constitutional logic should remain intentionally small:

```
participate = (Peace OR Clarity OR ServiceToOthers) AND NOT AbsoluteProhibition
```

The implementation should use explicit policy rules before any model-assisted interpretation. Model judgment may help classify ambiguity but must not be the sole authority for irreversible or high-risk access decisions.

8.1 Recommended output contract

```
type LambDecision = {
  decision: "PASS" | "REFUSE" | "HOLD_UNCERTAIN";
  peace: boolean | "unknown";
  clarity: boolean | "unknown";
  service: boolean | "unknown";
  prohibition_hits: string[];
  policy_version: string;
  reasons: string[];
};
```

8.2 Hard rules

- Fail closed on explicit prohibited abuse.
- Do not grant new technical capability; Lamb Lens only permits or refuses participation.
- Version policies and include policy version in receipts.
- Do not unnecessarily store rejected sensitive payloads; hash/minimize where possible.
- Never let repeated requests create authority by recurrence.

9. FragGate and Capability Model

FragGate should be the single meaningful public execution door. Catalog/status/documentation can stay public, but any operation with side effects, protected state, privileged network access, model mutation, messaging, or local execution must resolve through FragGate.

9.1 Capability classes

Capability	Meaning
PUBLIC_READ	Read public catalog/status/documentation.
PUBLIC_ANALYZE	Run explicitly safe stateless analysis.

SESSION_EXEC	Execute a declared engine operation in a session.
STATE_WRITE	Write protected application/session state.
MODEL_UPDATE	Change model/weights/learning state.
NODE_JOIN	Join distributed node fabric.
TRUST_UPDATE	Modify another node's trust metadata; never self-granted.
NETWORK_EGRESS	Perform bounded outbound network access.
MESSAGE_SEND	Transmit a message through communications domain.
LOCAL_EXEC	Perform local process/filesystem/system execution.
RECOVERY_ACTION	Apply an explicit forward corrective/restoration action.
OPERATOR	Administrative authority; still subject to no-rollback law.

Capability attenuation invariant: C(next) is a subset of C(current). A downstream component may remove permission; it may never silently create permission.

10. SweepGate + Sentinel

SweepGate remains a structural sieve and airlock. Sentinel is the broader integrity monitor. They are complementary and should not be conflated with ethical authorization.

10.1 SweepGate duties

- Blocked sensitive keys and explicit off-origin URL handling.
- Structural poison/malware signatures as defense-in-depth.
- Isolation signal and digest generation.
- Payload/type/size validation before expensive parsing.

10.2 Sentinel duties

- Input PASS / SUSPECT / QUARANTINE / REJECT.
- Model/state poisoning and drift monitoring.
- Output tamper/integrity checks.
- Fail-closed halt/hold/isolate behavior.
- No rollback. Recovery means forward corrective action under RoseClock.

10.3 Required hardening beyond signatures

- Maximum request body size before request.text()/JSON parse.
- Archive/decompression ratio limits.
- MIME/type verification and parser sandboxing for hostile media.
- CPU, memory and wall-time budgets per domain.
- Outbound network allowlists and DNS/IP SSRF protections for networked domains.
- Strict schemas with additionalProperties=false on privileged operations.

11. Provenance / Input Packet

AION's Input Packet is retained because it solves a real architectural need: normalize the provenance that enters the system.

```
type InputPacket = {
  packet_id: string;
  source_id: string;
  source_type: string;
  observed_at?: string;
  received_at: string;
```

```
  content_hash: string;
  content?: unknown;              // minimized or omitted for sensitive flows
  metadata: Record<string, unknown>;
  trust_level: "UNTRUSTED" | "LOW" | "NORMAL" | "HIGH" | "CUSTODIAL";
  tags: string[];
  claims?: Array<{ claim:string; evidence_refs:string[]; confidence?:number }>;
};
```

Source trust is metadata, not truth. A high-trust source can still be wrong. ChainLock proves preservation of the packet; it does not prove the source claim is factually correct.

12. DecisionGATE

DecisionGATE remains a contextual policy/responsibility filter rather than a security authorization system. Its existing Definition / Evidence / Impact / Integrity / Responsibility structure is useful, but capability checks must occur separately.

- Definition: operation/claim is concrete enough to evaluate.
- Evidence: required evidence references exist for consequential claims.
- Impact: positive and negative consequences are represented where relevant.
- Integrity: action does not contradict declared constraints/policy.
- Responsibility: accountable actor/capability is identifiable.

DecisionGATE may return REVISE/BLOCK, but a PASS never grants a missing capability.

13. AZPIPE

AZPIPE is the internal routing/security fabric. Current 1.6.15 code locks the hop order and retains FoldLock fld3-wire as an internal mechanism. The target should keep the concept of a locked legal route while updating the public strip to include the new constitutional components.

13.1 Router contract

```
route(envelope):
  require envelope.fraggate_resolved == true
  require envelope.chainlock_in_hash
  require decisiongate in {PASS, approved_variant}
  require domain_id
  require capability_digest
  assert capabilities only decrease
  return invokeDomainDoor(domain_id, envelope)
```

13.2 Forbidden router behavior

- No direct engine execution from arbitrary slug without domain resolution.
- No forwarding the global operator bearer to unrelated upstream workers.
- No unrestricted service-binding access exposed to an engine.
- No changing public hop order based on product name.
- No silent fallback path that bypasses Lamb Lens / FragGate / ChainLock.

14. Domain Door Architecture

Domain	Representative software	Default policy
Vault / Custody	ARK; EmbryoLock	Network deny; external fetch deny; cross-engine IPC deny; dynamic code deny; capability-only secret access;

		minimal retained payload; strict auth.
Media / Authenticity / Physics	VibeLock; VeilLock; SpectralLock; TrajectoryLock	Hostile parser sandbox; MIME validation; decode/decompression caps; temp storage; network deny by default; CPU/memory budgets.
Evidence / Provenance	EmployeeLock; WhistleLock; PeaceLock; ShadowLock; M.I.A.Lock; ChronoLock	Evidence attribution, confidence and provenance; no autonomous accusation; public-record sourcing controls.
Language / Structure / Pattern	CodeLock; FoldLock; GlossaFilter; Zion Pattern Solver; GodLock; AZ-CLCE	Stateless analysis by default; corpus provenance; no truth authority from pattern score.
AI / Agent Cognition	AZAI; AZBot; AZHub; AZInterface	Model isolation; explicit tool capabilities; no inherited browser/network capability; prompt/context boundaries.
Research / Knowledge	AZBrowser; Aziel Corpus	Controlled network egress; URL allow/deny policy; SSRF defense; source citation; ingestion/OCR/media capabilities separate from browsing.
Communications	AZMail	Recipient/content policy; anti-abuse/rate controls; no implicit deanonymization; separate send capability.
Network / Connectivity	AZNet; MirageGrid; AzielTether	Network-control integrity only; no generic arbitrary networking; bounded peers and radios default off.
System / Local Execution	AZ-OS	Standalone privileged domain; filesystem/process/shell controls; local/privileged only; strongest capability boundary.
Simulation / Game	Post-King Chess	Low-risk isolated state; no inherited production capabilities.
Core Fabric	ChainLock; TemporalLock; StaticClock; RoseClock	Infrastructure primitives, not ordinary product engines.
Security Fabric	Lamb Lens; FragGate; SweepGate; Sentinel; DecisionGATE; AZPIPE	Participation, admission, integrity, authorization and routing.

15. Isolated Engine Contract

```
type EngineManifest = {
  slug: string;
  domain: string;
  version: string;
  operations: Record<string, {
    method: "GET" | "POST";
    required_capabilities: string[];
    side_effects: string[];
    network: "DENY" | "ALLOWLIST" | "CONTROLLED";
    max_input_bytes: number;
    max_output_bytes: number;
    timeout_ms: number;
    state_namespace?: string;
    schema_id: string;
  }>;
};
```

Every operation must declare its side effects and resource requirements before execution. Domain policy may further reduce them.

15.1 Engine sandbox requirements

- No access to secrets not explicitly bound to that engine/operation.

- No access to another domain's KV/DO/storage namespace.
- No generic fetch unless network capability is granted and constrained.
- No dynamic eval/function construction in hosted paths unless a dedicated code sandbox explicitly owns that risk.
- No unrestricted child process or shell in hosted Runtime.
- Structured output and bounded response size.

16. StaticClock

StaticClock is retained as a fixed time/condition reference. It is not the forward gear. It may stamp a condition or external time anchor relevant to a RoseClock transition, but cannot move or rewind the action state.

```
type StaticClockRecord = {
  id: string;
  observed_at: string;
  condition_set: Record<string, boolean | string | number>;
  validity: "PASS" | "FAIL" | "INCOMPLETE";
  source_refs: string[];
  hash: string;
};
```

17. ASE - Perspective Integrity

ASE is warranted as an optional service for consequential analytical workflows. It should run before a downstream engine treats an observation set as sufficiently framed.

```
type PerspectiveState = {
  integrity_score?: number;
  missing_angles: string[];
  bias_flags: string[];
  competing_views: string[];
  observation_status: "SUFFICIENT" | "INCOMPLETE" | "CONFLICTED";
  evidence_refs: string[];
};
```

- ASE cannot override Lamb Lens or capability policy.
- ASE cannot convert a perspective score into factual truth.
- ASE output should remain inspectable and evidence-linked.

18. VECTOR - Optional Direction Selection

VECTOR is warranted only where a domain genuinely needs to select among multiple allowed actions. It is not mandatory in the base Runtime path.

```
type VectorOutput = {
  candidates: string[];
  selected?: string;
  action_state: "EXECUTE" | "HOLD" | "AVOID" | "MONITOR";
  rationale_refs: string[];
  required_rose_transition?: string;
};
```

VECTOR proposes direction. RoseClock governs the actual forward state transition. DecisionGATE/capabilities govern whether the proposed action is allowed.

19. Oracle - Append-Only Learning

Oracle's useful core is continuous monitoring, comparison against prior state, drift detection, feedback integration and adaptive reweighting. Under this master architecture, every change to model state or memory becomes a RoseClock forward action.

```
type LearningDelta = {
  delta_id: string;
  model_id: string;
  previous_model_hash: string;
  evidence_refs: string[];
  expected_outcome?: unknown;
  actual_outcome?: unknown;
  deviation_score?: number;
  proposed_changes: Record<string, unknown>;
  authority: string[];
  rose_transition_hash: string;
  new_model_hash: string;
};
```

- No in-place model overwrite without preserving prior hash.
- No self-authorized trust increase.
- Training/weight changes require MODEL_UPDATE.
- Observed corrections are new forward learning states.

20. Constellation - Distributed Layer

Constellation should be added only when multiple genuinely independent nodes exist. It sits above completed node results so consensus cannot contaminate the independence it is supposed to measure.

```
NODE A --complete local chain--\
NODE B --complete local chain----> CONSTELLATION -> agreement/divergence record
NODE C --complete local chain--/
```

20.1 Node contract

```
type NodeProfile = {
  node_id: string;
  software_version: string;
  policy_version: string;
  trust_score: number;
  accuracy_history: number[];
  status: "SYNC" | "DESYNC" | "ISOLATED";
  public_key: string;
};
```

20.2 Consensus contract

```
type ConsensusRecord = {
  decision_id: string;
  input_digest: string;
  node_results: Array<{node_id:string; result_hash:string; confidence?:number}>;
  agreement: number;
  divergence_flags: string[];
  rule: "MAJORITY" | "WEIGHTED" | "UNANIMITY";
  output: unknown;
  statement: "AGREEMENT_EVIDENCE"; // never "truth"
  rose_transition_hash: string;
};
```

- A node never updates its own trust score.
- Trust updates require independent authority and TRUST_UPDATE capability.

- Isolation and reintegration are forward actions, not rollback.
- Consensus must not replace minority/disagreement evidence.

21. ForgeReceipt

ForgeReceipt is not the integrity authority. It renders the authoritative evidence produced by the fabric.

```
type RuntimeReceipt = {
  receipt_id: string;
  request_digest: string;
  provenance_digest?: string;
  lamb_decision?: string;
  fraggate_operation: string;
  decisiongate_state: string;
  domain: string;
  engine: string;
  rose_before: string;
  rose_transition: string;
  rose_after: string;
  staticclock_ref?: string;
  temporallock_hash?: string;
  chainlock_in_hash: string;
  chainlock_out_hash: string;
  result_hash: string;
  capability_digest: string;
  policy_versions: Record<string,string>;
  created_at: string;
};
```

22. Authentication, Authorization and Capability Security

The current Runtime uses one optional RUNTIME_TOKEN for session mutation. That is adequate as a simple current deployment gate but is too broad for the target architecture once model updates, node joins, trust updates, messaging, local execution and recovery-forward actions exist.

22.1 Target token/capability design

- Use scoped bearer/capability tokens or signed capability envelopes.
- Bind token to audience/domain/operation where practical.
- Short-lived session execution capabilities.
- Separate OPERATOR from MODEL_UPDATE, TRUST_UPDATE, LOCAL_EXEC, MESSAGE_SEND and NODE_JOIN.
- Never forward an operator credential to a product worker unless that worker is the explicit credential audience.
- Receipt should carry a capability digest, not secret token material.

22.2 Read authorization

Session status and receipt reads should be capability-bound if they can expose private execution metadata. High-entropy IDs are not a substitute for authorization.

23. Request Parsing and Resource Limits

Surface	Required control
HTTP body	Reject over configured byte cap before full text/JSON parse.
JSON	Depth/property/string-size limits; privileged schemas strict.

Media	MIME validation, pixel/duration limits, archive/decompression ratio caps.
Network	DNS/IP validation, private/reserved-address block, redirect revalidation.
Execution	CPU/memory/time budgets by operation.
Receipts	Bound count and individual receipt size.
Logs	No secrets/raw tokens; minimize sensitive content.
State	Per-domain namespace and retention policy.

24. Rate Limiting and Abuse Controls

The current in-memory per-isolate rate limiter is useful locally but is not sufficient as global distributed enforcement. Target enforcement should use a Durable Object, provider-native rate limiting/WAF, or another consistent distributed mechanism.

- Rate by authenticated principal + IP/risk signals, not only IP.
- Different limits for read, analysis, session open, exec, messaging, network, model update and node join.
- Return Retry-After and stable error code.
- Do not rely on X-Forwarded-For when outside a trusted proxy boundary.

25. State Storage

Persistent state must be namespaced by domain and classified by sensitivity.

State class	Recommended backing	Rules
RoseClock tips/transitions	Durable transactional store / DO	Atomic compare-and-swap; no overwrite of accepted transitions.
ChainLock	Append-only durable store	Verify previous hash on write; immutable stamps.
TemporalLock	Append-only durable store	Ordered evidence; stable clock/source metadata.
Session state	Durable Object	TTL, auth, bounded receipts, private read capability.
Oracle learning	Versioned model/memory store	Prior hash + delta + new hash + authority.
Constellation	Signed node/consensus records	Independent node signatures; trust updates separately authorized.
Public counters	KV/provider analytics	Never use eventual-consistency counters as security authority.

26. Canonicalization and Hashing

- Use one canonical JSON implementation across all integrity primitives.
- Exclude only explicitly non-authoritative fields from hash input.
- Include version/schema identifiers in canonical objects.
- Hash raw bytes for files/media; hash canonical object for structured envelopes.
- Never hash a truncated display representation as the only authoritative content hash.
- Use SHA-256 minimum for existing compatibility; design interfaces so algorithm version can migrate.
- Every receipt names hash algorithm and schema version.

27. Replay and Idempotency

Forward-only state requires replay protection. A valid old signed request must not be executable indefinitely.

```
ExecutionEnvelope {
  request_id,
  nonce,
  issued_at,
  expires_at,
  expected_rose_tip,
  operation,
  payload_digest,
  capability_digest
}
```

- Reject duplicate request_id/nonce for state-changing operations.
- Bind state-changing call to expected RoseClock tip.
- Provide idempotency key semantics for safe client retries.
- Replayed read-only analysis may be allowed if explicitly stateless.

28. Error Model

Code	Meaning
ETHICAL_REFUSE	Lamb Lens prohibition or no qualifying participation basis.
UNKNOWN_OPERATION	FragGate registry miss.
CAPABILITY_DENIED	Required capability absent.
INPUT_QUARANTINED	SweepGate/Sentinel isolated input.
BODY_TOO_LARGE	Pre-parse size cap exceeded.
POLICY_BLOCK	DecisionGATE block.
DOMAIN_DENIED	Domain policy disallows resource/action.
ROSE_CONFLICT	Expected RoseClock tip is stale.
ROSE_INVALID_TRANSITION	Backward/invalid transition.
TEMPORAL_INCOMPLETE	Required timing evidence missing.
CHAIN_VERIFY_FAIL	Hash continuity failure.
RATE_LIMIT	Distributed abuse limit reached.
STATE_WRITE_CONFLICT	Atomic state update conflict.

29. API Shape

29.1 Public execution

```
POST /v2/call
Authorization: Bearer <scoped capability>
Idempotency-Key: <uuid>

{
  "operation": "product.verb",
  "input_packet": {...},
  "expected_rose_tip": "...",
  "options": {...}
}
```

29.2 Response

```
{
  "ok": true,
  "result": {...},
  "state": {
```

```
    "rose_transition": "...",
    "rose_tip": "..."
  },
  "receipt": {...}
}
```

29.3 Public non-executable surfaces

- GET catalog / software manifests
- GET health/readiness with no sensitive configuration leakage
- GET OpenAPI/skill documents
- GET public citations/download metadata

Any endpoint that can cause meaningful state change must not bypass the unified execution contract.

30. Suggested Repository Layout

```
aziel-runtime/
  src/
    gateway/
      lamb-lens.js
      fraggate.js
      request-limits.js
      auth.js
      capabilities.js
    security/
      sweepgate.js
      sentinel.js
      decisiongate.js
    provenance/
      input-packet.js
      source-policy.js
    fabric/
      azpipe.js
      domain-router.js
      capability-attenuation.js
    integrity/
      chainlock/
      temporallock/
      staticclock/
      roseclock/
      receipts/
    domains/
      vault/
      media/
      evidence/
      pattern/
      cognition/
      research/
      communications/
      network/
      local-exec/
      simulation/
    engines/
      <engine-slug>/
    analysis/
      ase/
      vector/
      oracle/
    distributed/
      constellation/
      node-trust.js
    schemas/
      *.schema.json
```

```
    storage/
      session-do.js
      rose-store.js
      append-store.js
    api/
      v2-call.js
      public-read.js
      mcp.js
    tests/
      unit/
      integration/
      adversarial/
      property/
      replay/
      concurrency/
      golden/
  docs/
    MASTER-ARCHITECTURE.md
    ROSECLOCK.md
    SECURITY.md
    THREAT-MODEL.md
```

31. Coding Rules

- Default deny: new operation is not callable until registered with domain, schema and capabilities.
- Strict input validation before domain execution.
- No additionalProperties on privileged schemas unless explicitly justified.
- No silent fallback from local engine to upstream proxy if the upstream path has weaker gates.
- No global fetch helper available to non-network domains.
- No global mutable operator token passed through internal envelopes.
- No raw secrets in ChainLock facts, TemporalLock records, receipts or observability logs.
- Every state-changing function accepts expected parent/tip and fails on conflict.
- Every correction/recovery function name must make forward semantics explicit.
- Never implement a function named rollback that mutates accepted RoseClock history.
- Every adaptive update records previous hash, delta/evidence, authority and new hash.
- Every distributed trust update identifies who authorized it; node cannot self-promote.
- Tests must assert capability attenuation at every hop.
- Hash and canonicalization functions are shared primitives, not reimplemented per engine.
- Domain policy is data/config plus code-enforced hard limits; UI labels are not security.

32. Security Tests Required Before Production

Test family	Acceptance condition
Hop-bypass tests	Attempt direct product worker, proxy, MCP, alias, session and internal-route bypass of Lamb Lens/FragGate/ChainLock.
RoseClock property tests	Random transition sequences must never decrease sequence or change accepted parent history.
Concurrency tests	Two writers on same tip; exactly one linear commit succeeds unless explicit branch mode.
Replay tests	Old nonce/request/state-changing envelope cannot execute twice.
Capability tests	No downstream hop increases capability; operator capability not forwarded accidentally.
SSRF tests	Networked domains cannot reach private/reserved/link-local

	metadata targets.
Body bomb tests	Oversized JSON/archive/media refused before expensive parse/decode.
Schema fuzzing	Unknown privileged fields rejected; type confusion fails closed.
Chain tamper tests	Modify fact/prev/hash and verification isolates chain.
Oracle poisoning tests	Adversarial feedback cannot mutate model without MODEL_UPDATE and evidence record.
Constellation tests	Colluding/minority/compromised nodes preserve disagreement evidence; no self trust update.
Receipt forgery tests	ForgeReceipt cannot manufacture authoritative chain hashes.
Secret logging tests	Tokens/private fields absent from logs/receipts/traces.

33. Architecture Migration Plan

1. Freeze a versioned MASTER-ARCHITECTURE contract and treat old AION rollback/ZD30-in-core language as historical.
2. Implement RoseClock as a standalone integrity/action-time primitive with property and concurrency tests.
3. Integrate RoseClock transition hash into ChainLock and TemporalLock records without changing their narrow roles.
4. Implement scoped capability envelope and attenuation checks; retain legacy RUNTIME_TOKEN only as temporary operator compatibility.
5. Add gateway body caps, strict schemas and distributed rate limiting.
6. Add Lamb Lens as the participation layer with policy versioning and explicit absolute prohibitions.
7. Add Sentinel quarantine/fail-closed monitoring; remove literal rollback states from Sentinel language/code.
8. Normalize provenance through InputPacket before ChainLock-IN.
9. Refactor AZPIPE/domain router so engines only receive declared bindings/capabilities.
10. Split or wrap high-risk domains so AZ-OS, browser/network, messaging, media parsing and vault/custody cannot share ambient privileges.
11. Make receipt/status reads capability-aware where private metadata exists.
12. Add Oracle only after model/memory state is versioned append-only.
13. Add Constellation only after independent node execution, signatures and trust governance exist.

34. Source Reconciliation

The current Runtime 1.6.15 source already locks a public hop order and identifies SweepGate, ChainLock, DecisionGATE, AZPIPE, domain doors, TemporalLock, StaticClock and ChainLock-OUT. The target specification keeps those components while adding the clarified RoseClock and selected AION-derived services.

The AION papers support retaining: Input Packet/provenance, ASE perspective integrity, Oracle adaptive monitoring/learning, Sentinel quarantine/fail-closed defense, and Constellation independent-node comparison. Their older ZD30-centered and rollback language is not imported into the constitutional Runtime.

RoseClock's older description as recurrence/cycle recognition is preserved as a useful subfunction, but the current master definition is stronger: RoseClock is the forward-only gear-based action-time state machine.

> **This document is the controlling design specification when older papers conflict with the forward-only RoseClock law.**

35. Final Architecture Summary

```
ETHICS
  Lamb Lens

PUBLIC EXECUTION
  FragGate

INPUT / INTEGRITY
  SweepGate + Sentinel
  Provenance InputPacket
  ChainLock-IN

POLICY / ROUTING
  DecisionGATE
  AZPIPE
  Domain Door

EXECUTION
  Isolated Engine

ACTION-TIME
  RoseClock  [FORWARD ONLY]
    + StaticClock evidence when needed
    + TemporalLock evidence/hash when needed

OUTCOME
  ChainLock-OUT
  ForgeReceipt
  Return

OPTIONAL ANALYTICAL SERVICES
  ASE      = perspective integrity
  VECTOR   = directional selection
  Oracle   = append-only learning
  Constellation = independent multi-node comparison

ABSENT FROM CONSTITUTIONAL CORE
  ZD30
  rollback
  generic truth score
```

Final law: systems may change state, correct state, restore configuration, revoke authority, quarantine a compromise, or learn from error. Every one of those is another forward action. RoseClock never rolls backward; ChainLock and TemporalLock simply produce more verifiable record.

Appendix A - Minimal Build Interfaces

```
interface RoseClock {
  getTip(roseId: string, branchId?: string): Promise<RoseState>;
  propose(current: RoseState, input: RoseTransitionInput): Promise<RoseTransition>;
  validate(current: RoseState, transition: RoseTransition, caps: CapabilitySet): Promise<void>;
  commit(expectedTipHash: string, transition: RoseTransition): Promise<CommitResult>;
}

interface ChainLock {
  append(chain: string, fact: CanonicalFact, refs?: IntegrityRefs): Promise<ChainStamp>;
  verify(chain: string): Promise<VerifyResult>;
}

interface TemporalLock {
```

```
  append(event: TemporalEvent): Promise<TemporalStamp>;
}

interface LambLens {
  evaluate(request: NormalizedRequest, policyVersion: string): Promise<LambDecision>;
}

interface FragGate {
  resolve(operation: string, presentedCapabilities: CapabilitySet): Promise<ResolvedOperation>;
}

interface DomainDoor {
  attenuate(capabilities: CapabilitySet, manifest: EngineManifest): CapabilitySet;
  execute(envelope: ExecutionEnvelope): Promise<EngineResult>;
}
```

Appendix B - Transaction Skeleton

```
async function runtimeCall(req, env) {
  const bounded = await parseBoundedRequest(req);

  const lamb = await lambLens.evaluate(bounded);
  if (lamb.decision !== "PASS") return refuse(lamb);

  const op = await fragGate.resolve(bounded.operation, bounded.capabilities);

  const sweep = await sweepGate.inspect(bounded);
  const sentinel = await sentinel.inspect(bounded, sweep);
  if (sentinel.blocking) return quarantineOrRefuse(sentinel);

  const packet = await provenance.normalize(bounded);
  const inStamp = await chainLock.append("session", ingressFact(packet, op));

  const decision = await decisionGate.check(packet, op);
  if (decision.state !== "PASS") return reviseOrBlock(decision, inStamp);

  const routed = await azpipe.route({
    op, packet, inStamp,
    capabilities: attenuate(bounded.capabilities, op.required_capabilities)
  });

  const result = await routed.domainDoor.execute(routed);

  const currentRose = await roseClock.getTip(routed.rose_id);
  const proposed = await roseClock.propose(currentRose, {
    action: op.name,
    result_hash: hash(result),
    authority: routed.capabilities
  });
  const roseCommit = await roseClock.commit(currentRose.state_hash, proposed);

  const temporal = op.requires_temporal_evidence
    ? await temporalLock.append(fromRose(roseCommit))
    : null;

  const staticRecord = op.requires_static_conditions
    ? await staticClock.record(op.static_conditions)
    : null;

  const outStamp = await chainLock.append("session",
    outcomeFact(result, roseCommit, temporal, staticRecord));

  return forgeReceipt.render({
    result, lamb, op, inStamp,
    roseCommit, temporal, staticRecord, outStamp
```

```
  });
}
```

Appendix C - Implementation Acceptance Checklist

- RoseClock cannot decrement or overwrite accepted state.
- Restore/correct/revoke paths create new RoseClock transitions.
- ChainLock/TemporalLock never expose rollback APIs.
- State-changing calls require expected RoseClock tip.
- All privileged operations declare domain and capabilities.
- Capabilities monotonically attenuate.
- All request parsing is size-bounded before full parse.
- Private session receipt reads require authorization.
- Distributed rate limiting replaces per-isolate-only enforcement.
- Network-capable domains have SSRF/egress controls.
- Vault/local-exec/media/communications domains do not inherit unrelated bindings.
- Sentinel contains and advances forward; no rollback state.
- Oracle learning is versioned and hash-linked.
- Constellation runs only over independent completed node outputs.
- Consensus labels agreement, not truth.
- ForgeReceipt renders but cannot forge authoritative integrity state.
- ZD30 is absent from the constitutional core.